Exact Metric Dimensions and Improved Upper Bounds for Hypercubes

Shmulik Cohen

August 2026

Abstract

In this paper we consider the problem of determining the metric dimension of the hypercube Q_n. Exact values are difficult to obtain even for modest dimensions, and the previously established sequence stopped at $n = 13$. We prove that $\beta(Q_{14}) = \beta(Q_{15}) = 9$. The lower bound is obtained by an exhaustive enumeration of detecting matrices, using hypercube symmetries and orderly generation to reduce the search. The computation is implemented separately in C and safe Rust and is supported by direct checks on smaller instances and comparisons with independent data. As a consequence we also determine the nonadaptive coin-weighing value $M(14) = 8$. Finally, we give explicit, independently checkable matrices that improve the known upper bounds to $\beta(Q_{24}) \le 13$, $\beta(Q_{26}) \le 14$, and $\beta(Q_{29}) \le 15$.

1 Introduction

The vertices of the hypercube Q_n are the binary vectors $\{0,1\}^n$, with two vertices adjacent when they differ in one coordinate. A set R of vertices is *resolving* if the vectors of Hamming distances

$$\big(d(x,r) : r \in R\big), \qquad x \in \{0,1\}^n,$$

are pairwise distinct. The minimum cardinality of a resolving set is the metric dimension $\beta(Q_n)$.

The metric-dimension problem was introduced independently by Slater [16] and Harary and Melter [6]. For hypercubes it is closely related to nonadaptive coin weighing and detecting matrices [4, 2, 15, 9]. Its asymptotic order is known, but exact small values are difficult. The public sequence A303735 records exact values through $n = 13$ as

$$1, 2, 3, 4, 4, 5, 6, 6, 7, 7, 8, 8, 8$$

[13]. The smaller terms have since been independently confirmed: V. Miller verified the first ten values with a MaxSat solver and symmetry-breaking constraints [13]. Heuristic searches had already produced the upper bounds $\beta(Q_{14}) \le 9$ and $\beta(Q_{15}) \le 9$ [11, 12, 7]. What remained open was a lower bound ruling out eight landmarks.

This paper supplies that lower bound by exhaustive computation. The main result is the following.

Theorem 1. *The following statements hold.*

(i) The metric dimensions of the 14- and 15-dimensional hypercubes are

$$\beta(Q_{14}) = \beta(Q_{15}) = 9.$$

(ii) The metric dimensions of the 24-, 26-, and 29-dimensional hypercubes satisfy

$$\beta(Q_{24}) \le 13, \qquad \beta(Q_{26}) \le 14, \qquad \beta(Q_{29}) \le 15.$$

The computation uses familiar hypercube symmetries. Nikolić et al. showed, among other reductions, that a resolving set may be translated to contain the zero vertex and that complementing landmarks preserves resolving pairs [12]. Their implementation used these facts in a greedy upper-bound heuristic and used lexicographic order for candidate tie-breaking. Our lower-bound method differs in kind: we combine the normalized detecting-matrix formulation with an exhaustive sorted-prefix search, rejecting a prefix whenever it is not the lexicographically least member of its symmetry orbit. The prefix rule is proved in Section 4; it is established directly, not inferred from heuristic success.

The code, certificates, complete correctness argument, and reproduction scripts are available in the accompanying repository [3]. The exact claim does not depend on the stochastic searches used to discover the upper bounds.

2 Resolving sets and detecting matrices

Let $W \in \{-1, +1\}^{q \times n}$. We call W a *detecting matrix* if

$$Wz \neq 0 \quad \text{for every nonzero } z \in \{-1, 0, 1\}^n.$$

Lemma 2. *There is a resolving set of size q in Q_n if and only if there is a $q \times n$ detecting matrix.*

Proof. Given landmarks $v_1, \dots, v_q \in \{0,1\}^n$, set $W_{ij} = 1 - 2(v_i)_j$. For $x, y \in \{0,1\}^n$, direct expansion gives

$$d(x, v_i) - d(y, v_i) = \sum_{j=1}^{n} (x_j - y_j) W_{ij}.$$

Thus two distinct vertices have identical distance vectors precisely when the nonzero vector $z = x - y \in \{-1, 0, 1\}^n$ lies in the kernel of W. Conversely, every nonzero $z \in \{-1, 0, 1\}^n$ can be written as $x - y$ for binary $x \neq y$, so the implication reverses. □

Multiplying a column of W by -1 preserves the condition: if D is a diagonal sign matrix, then $WDz = 0$ if and only if $W(Dz) = 0$, and D permutes $\{-1, 0, 1\}^n$. We may therefore flip columns so that the first row is all $+1$.

Lemma 3 (subset-sum form)**.** *A sign matrix is detecting if and only if the 2^n subset sums of its columns are pairwise distinct.*

Proof. Two equal subset sums, indexed by $A, B \subseteq [n]$, give a ternary kernel vector after canceling $A \cap B$: assign $+1$ to $A \setminus B$, -1 to $B \setminus A$, and zero elsewhere. Conversely, the positive and negative supports of any ternary kernel vector give two equal subset sums. □

In the language of additive combinatorics, Lemma 3 says that the columns of a detecting matrix form a *dissociated* set: a set of vectors with no nontrivial $\{-1, 0, 1\}$-combination summing to zero, equivalently a set whose subset sums are pairwise distinct [1]. The metric dimension $\beta(Q_n)$ is thus the least number of coordinates q in which n sign vectors can be dissociated; Theorem 1(i) states that no fourteen vectors in $\{-1, +1\}^8$ are dissociated, whereas fifteen can be dissociated in nine coordinates. Because the first row is normalized to all $+1$, any two equal subset sums must

have equal cardinality; under the coordinate bijection $u \mapsto \frac{1-u}{2}$, collisions in $\{-1,+1\}^{q-1}$ between subsets of the same size map bijectively to $\{0,1\}^{q-1}$ subset-sum collisions. In this $\{0,1\}$-valued form, maximal systems are catalogued as maximum dissociated sets of 0–1 vectors [10], which is why those catalogues furnish an independent check in Section 6.

After normalization, a column is determined by its last $q-1$ signs. We encode it as a point of $\{0,1\}^{q-1}$, so there are only 2^{q-1} column types. Repeated columns are impossible, since their difference is a nonzero ternary kernel vector. The search object is therefore an n-subset of a 2^{q-1}-element type set.

A small example illustrates the role of the normalized row. For four columns, write

$$W = \left[\begin{array}{l|cccc} & c_1 & c_2 & c_3 & c_4 \\ \hline \text{normalized row} & +1 & +1 & +1 & +1 \\ & +1 & -1 & +1 & -1 \\ & -1 & +1 & +1 & -1 \end{array}\right].$$

The first coordinate of a subset sum is its cardinality. Hence subsets of different sizes cannot collide, while the remaining rows must distinguish subsets of the same size. Normalization does not itself establish the detecting property; it exposes a canonical coordinate and reduces each column to one of 2^{q-1} types.

3 Incremental exhaustive search

Represent a partial object by an increasing list of types $P = (c_1 < \cdots < c_k)$. Let

$$\Sigma(P) = \left\{ \sum_{c \in A} c : A \subseteq P \right\}$$

denote its subset sums in the original sign-vector coordinates. If all sums in $\Sigma(P)$ are distinct, adjoining a candidate c is legal exactly when

$$\Sigma(P) \cap \big(\Sigma(P) + c\big) = \varnothing. \tag{1}$$

The child sums are then the disjoint union of these two sets. The enumerator stores the sums once per depth and tests (1) with a hash table.

Without symmetry pruning, the depth-first traversal is complete: at depth k, every legal larger candidate is tried in increasing order and only the suffix after that candidate is passed to the child. Thus every collision-free sorted type set containing the fixed first type appears once.

Two elementary pruning rules are monotone.

(i) If a candidate c is illegal for P, equation (1) remains false for every extension of P, because the witnessing subset sums remain present.

(ii) If fewer than $n-k$ live candidates remain, the prefix cannot reach n columns.

For the decisive 8×14 case, each coordinate of a subset sum is packed into one byte of a 64-bit word with bias 64. With at most 14 columns every coordinate lies between 50 and 78, so no carry or borrow can cross a byte boundary. The packed addition is therefore exact componentwise arithmetic, not a probabilistic hash.

4 Orderly generation and prefix pruning

The full automorphism group acting on $q \times n$ sign matrices is the signed permutation group $O(q, \mathbb{Z}) = (\mathbb{Z}_2)^q \rtimes S_q$ on the rows (of order $2^q q!$), combined with column permutations S_n and column sign flips $(\mathbb{Z}_2)^n$. Fixing the first row to all $+1$ by column sign flips reduces each column to one of 2^{q-1} normalized column types $V = \{w \in \{-1, +1\}^q : w_1 = +1\} \cong \{0, 1\}^{q-1}$.

For any hypercube isometry $g \in O(q, \mathbb{Z})$ acting on $\{-1, +1\}^q$, we define its normalized action $\Phi_g : V \to V$ by

$$\Phi_g(w) = (g(w))_1 \cdot g(w).$$

Because $(g(w))_1 \in \{-1, +1\}$, the first coordinate of $\Phi_g(w)$ is identically $+1$. The map Φ_g is a pointwise bijection on the finite set V: if $\Phi_g(u) = \Phi_g(v)$, then $g(u) = \sigma g(v) = g(\sigma v)$ for $\sigma = (g(u))_1 (g(v))_1 \in \{-1, +1\}$; invertibility of g gives $u = \sigma v$, and comparing first coordinates yields $u_1 = +1 = \sigma v_1 = \sigma$, so $u = v$.

For group elements fixing the first row, $g \in \mathrm{Stab}(\text{row } 1) \cong S_{q-1} \ltimes (\mathbb{Z}_2)^{q-1}$, the action on $\{0, 1\}^{q-1}$ reduces to

$$x \longmapsto \pi(x \oplus t),$$

where $\pi \in S_{q-1}$ permutes the lower coordinates and $t \in \{0, 1\}^{q-1}$ is a translation. For elements that pivot a different row $R \in \{1, \ldots, q-1\}$ to row 1, Φ_g acts on the type bits $(b_0, \ldots, b_{q-2}) \in \mathbb{F}_2^{q-1}$ as the invertible linear map

$$\mathrm{bit}_{\text{new Row } 0}(t) = b_{R-1}, \qquad \mathrm{bit}_{\text{new Row } r'}(t) = b_{R-1} \oplus b_{r'-1}.$$

We sort every type set lexicographically. Translation allows one chosen member of a set to become zero, so every orbit has a sorted representative containing zero; the search fixes zero as its first type. At a partial set P, the program tests the normalized group actions and rejects P if a sorted image is lexicographically smaller.

Proposition 4 (canonical-prefix rule)**.** *If a sorted full set $X \subset V$ is lexicographically least in its orbit under the normalized group action, then every initial prefix $A \subseteq X$ is lexicographically least among its normalized sorted images $\Phi_g(A)$ that contain zero. Consequently, rejecting a nonminimal prefix cannot remove the last representative of a solution orbit.*

Proof. Let A be the first k elements of X. Because $\Phi_g : V \to V$ is a pointwise bijection, $\Phi_g(A) \subseteq \Phi_g(X)$. Let sorted $\Phi_g(A) = (y_1 < \cdots < y_k)$ and sorted $\Phi_g(X) = (z_1 < \cdots < z_n)$. Since $\Phi_g(A)$ is a k-subset of $\Phi_g(X)$, the j-th smallest element of the superset is at most the j-th smallest element of the subset: $z_j \le y_j$ for all $1 \le j \le k$.

Suppose some group element g yields a normalized sorted image $\Phi_g(A)$ strictly lexicographically smaller than A. Let $m \in \{1, \ldots, k\}$ be the first index where $y_m < x_m$ (with $y_j = x_j$ for $j < m$). For all $j < m$, $z_j \le y_j = x_j$. If $z_j < x_j$ for any $j < m$, then sorted $\Phi_g(X) <_{\mathrm{lex}} X$. If $z_j = x_j$ for all $j < m$, then at index m, $z_m \le y_m < x_m$, which again yields sorted $\Phi_g(X) <_{\mathrm{lex}} X$. In either case, X was not lexicographically minimal in its orbit, a contradiction. □

This is an instance of Read–Faradžev orderly generation [14, 5]. For $q = 8$, the stabilizer group has order $7!\,2^7 = 645{,}120$, while the full row group has order $8!\,2^8 = 10{,}321{,}920$. The baseline enumeration canonicalizes under the stabilizer through depth eight, which is already sufficient to complete the search deterministically. Stopping early may repeat work but cannot discard a solution.

The minimality test itself avoids running all $(q-1)!$ permutations on most prefixes. A permutation only reorders coordinates, so it preserves the popcount of every column type; the least

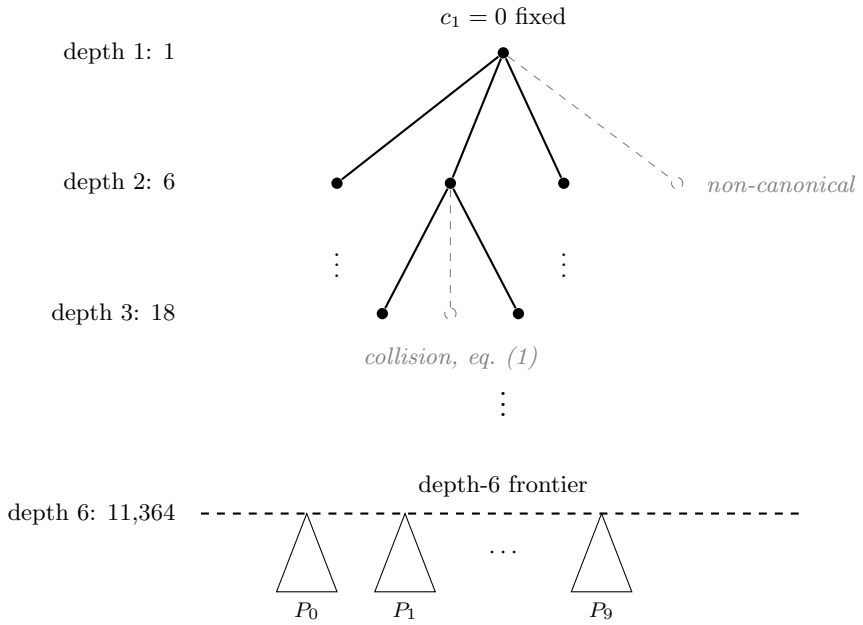

Figure 1: Schematic of the canonical-prefix enumeration for the 8×14 case. Each node is a sorted, collision-free prefix; depth is the number of columns selected, with the first type fixed to zero. Solid edges are explored; a dashed edge is pruned, either because the candidate would create a subset-sum collision (equation (1)) or because the prefix is not lexicographically least in its symmetry orbit (Proposition 4). The counts on the left are the numbers of canonical prefixes at each depth. The depth-six frontier of 11,364 prefixes is split into ten congruence classes $P_0, \ldots, P_9$ modulo 10, each searched independently and each reporting no 8×14 detecting matrix.

type index attainable for a type of popcount w is therefore fixed in advance. Comparing these popcount-derived bounds against the incumbent already settles the ordering for the large majority of prefixes, and the full permutation sweep is run only for the few that survive this cheap filter.

Figure 1 sketches the pruned enumeration and the depth-six frontier at which the computation is partitioned; the scale of the reduction is shown in Table 1. These counts come from a controlled 7×11 run and from the common 8×14 frontier; they describe performance, not an additional premise of correctness.

Table 1: Effect of canonical-prefix pruning.

Measurement	Without prefix test	With prefix test
7×11, complete search nodes	256,123,828	189,831
8×14, nodes at depth 5	1,282,411	1,082
8×14, nodes at depth 6	34,942,921	11,364

Here *depth* means the number of columns already selected. Thus depth six is the frontier of all legal canonical six-column prefixes, not six levels of a binary decision tree.

5 Partitioning the decisive computation

The 8×14 traversal is divided into ten processes at depth six. Every process deterministically walks the same tree down to that frontier. For each outgoing child it increments the same counter; partition p retains the child exactly when the counter is congruent to $p \pmod{10}$. Every outgoing branch therefore belongs to exactly one partition.

All completed logs have the identical frontier vector

$$(N_0, \ldots, N_6) = (0, 1, 6, 18, 143, 1082, 11364).$$

The ten reported totals sum to 340,092,619 node visits. The common 12,614 shallow nodes ($\sum_{i=0}^{6} N_i = 12{,}614$) are visited by every process, so nine duplicate copies must be removed. The number of distinct search-tree nodes is therefore

$$340{,}092{,}619 - 9 \cdot 12{,}614 = 339{,}979{,}093.$$

Each partition exits successfully and reports zero solutions.

The partition-log checker is conservative by design: it accepts a run only when the logs comprise exactly partitions 0 through 9, exactly one completed verdict per file, the common frontier above, and no emitted solution. Malformed, duplicate, missing, or interrupted logs are rejected.

6 Verification and computational results

The proof obligation is narrower than the observation that two independent programs reported no solution. It is that the implemented traversal covers every normalized object up to the proved group action, and that every accepted pruning rule preserves at least one representative. Sections 2–5 establish those facts. We used the following controls to test the implementation against those arguments.

Table 2: Verification map.

Control	Result
Direct Python enumeration	Exact raw solution sets agree on 3×4, 4×4, and 5×6.
Whole-orbit canonicalization	All 13 independently computed canonical 5×6 representatives agree.
Partition control	Three small-instance parts are disjoint and their union is the direct reference set.
Runtime instrumentation	AddressSanitizer and UndefinedBehaviorSanitizer pass the small complete comparisons.
Known-value ladder	The code reproduces $\beta(Q_{11}) = \beta(Q_{12}) = \beta(Q_{13}) = 8$.
Independent data	All 118,485 McKay dissociated 12-set orbits occur in the 8×13 output after the proved conversion; none is missing [10].
Second implementation	Safe Rust, with no external crates and no `unsafe`, matches every complete C node vector in all ten 8×14 partitions.

On the same arm64 machine, the C partitions took 350.60–372.36 seconds and the Rust partitions at most 365.57 seconds. These timings are descriptive and are not used in the proof.

The C and Rust programs deliberately implement the same orderly-generation argument. Their exact agreement is strong evidence against language-specific, state-management, and memory-safety errors, but it is not an independent mathematical method or a third-party verification. The natural route to a machine-checkable strengthening is a proof of nonexistence: encode *normalized detecting-matrix existence* as a Boolean satisfiability instance and, on an `UNSAT` verdict, validate the emitted DRAT refutation with an independent checker—or, after conversion to LRAT, with a formally verified checker such as `cake_lpr` [17], as done for the Boolean Pythagorean triples problem [8]. This succeeds on the small instances: we obtained checked refutations for every nonexistence case through 4×6 (hence $\beta(Q_6) \geq 5$). It does not reach the decisive scale. The instances are resolution-hard—a modern CDCL solver did not close 5×7 within minutes, whereas the enumerator settles it in under a second—and the direct 8×14 encoding already exceeds 6×10^7 clauses. Closing that size would require a substantially more compact encoding together with cube-and-conquer partitioning, which we record as the principal open direction for independent verification.

Proof of Theorem 1(i). The exhaustive computation finds no 8×14 detecting matrix, so Lemma 2 gives $\beta(Q_{14}) \geq 9$. The explicit 9×15 detecting matrix in Appendix A yields, by deleting any one of its columns, a 9×14 detecting matrix. Hence $\beta(Q_{14}) \leq 9$.

If an 8×15 detecting matrix existed, deleting any column would leave an 8×14 detecting matrix: a kernel vector for the smaller matrix could be extended by a zero coordinate. Thus $\beta(Q_{15}) \geq 9$, and the same explicit 9×15 matrix supplies the matching upper bound. □

7 New upper bounds

The second part of Theorem 1 is certified by the three sign matrices reproduced in Appendix A. Their sizes and the resulting bounds are summarized in Table 3.

Table 3: New upper-bound certificates.

Dimension	Rows	Previous upper bound	New upper bound
24	13	14	13
26	14	15	14
29	15	16	15

The previous values in the third column follow from the computational and constructive bounds reported by Nikolić et al., Hertz, and Lu–Ye [12, 7, 9]. Thus each certificate improves the corresponding published bound by one.

Proof of Theorem 1(ii). For each matrix W in Appendix A, the exhaustive meet-in-the-middle check of Section 7.1 finds no nonzero $z \in \{-1, 0, 1\}^n$ satisfying $Wz = 0$. Hence each matrix is detecting, and Lemma 2 gives the stated upper bound. Separate C and JavaScript verifiers produce the same result – one searching a sorted signature table, the other a hash table, so agreement is not an artifact of shared code – and both reject matrices with a deliberately duplicated column. □

The proof depends only on the displayed matrices and their finite verification, not on the heuristic search that produced them. Deleting columns also gives the corresponding bounds for all smaller dimensions, although those consequences do not improve the previously known values at dimensions 23, 25, 27, and 28.

7.1 Search methods

The certificates were discovered in a separate upper-bound search built around a single exact primitive: a meet-in-the-middle ternary-kernel counter. Splitting the n columns into two halves, the routine enumerates the $3^{\lceil n/2 \rceil}$ signed subset sums of one half, sorts their packed row-sum signatures, and streams the signed subset sums of the other half, counting exact cancellations $Wz = 0$. Each half contributes per-row sums of small magnitude, so a full signature packs into two machine words with one five-bit field per row and comparisons reduce to single instructions. This replaces the 3^n cost of naive ternary enumeration by $O\big(3^{\lceil n/2 \rceil}\big)$ time and space; even for the 15×29 matrix ($3^{15} = 14{,}348{,}907$ entries, occupying $\approx 115\,\mathrm{MB}$ of memory), the kernel count is computed exactly in a fraction of a second. The same primitive drives both the search and the final check, so a discovered matrix is re-examined by an exact count rather than a sampled estimate.

The 15×29 matrix was found by simulated annealing on the ± 1 entries. To keep each proposal cheap, the annealing objective is a *bounded-weight* kernel count: only ternary vectors with at most a fixed number of nonzero entries are enumerated. This underestimates the true count but is far faster, and whenever it reaches zero the full exact counter re-certifies the matrix; the exact null vectors it returns then steer the next move. Two further reductions exploit the meet-in-the-middle split. Because the two halves are independent, flipping one entry invalidates only one side's sorted table, so each accepted move rebuilds a single $3^{\lceil n/2 \rceil}$-sized side and reuses the other unchanged. When the search stalls, repairs are restricted to the coordinates in the support of the current null vectors, exploiting the structure of the near-certificate rather than its count alone.

The remaining certificates were obtained without a fresh anneal. The 14×26 matrix is a row-and-column submatrix of the annealed 15×29 solution whose kernel count, evaluated once, is already zero. The 13×24 matrix came from an *extension scan*: given a solved 13×23 submatrix, the two sorted half-tables are built once, and each of the 2^{13} candidate columns is then tested by a small update to one table followed by a single merge, keeping exactly the columns whose addition creates no ternary kernel vector. This reduces the cost of testing all 2^{13} extending columns from 2^{13} independent kernel counts to a single table build followed by 2^{13} inexpensive merges; two independent 13×23 slices extended by the same column, corroborating the result. These procedures explain how the witnesses were found, but they are not part of their proof.

8 Coin weighing

Let $M(n)$ be the minimum number of fixed $\{0, 1\}$-valued spring-scale weighings that distinguish all subsets of n coins. The standard relation [15, 9] is

$$\beta(Q_n) - 1 \le M(n) \le \beta(Q_n).$$

The left inequality follows by adjoining an all-ones row to a weighing matrix; the right follows from normalizing a detecting matrix. It is only an inequality, not an identity.

Corollary 5. $M(14) = 8$.

Proof. Theorem 1 with the left inequality above gives $M(14) \ge 8$. The artifact includes an 8×14 binary weighing matrix whose 2^{14} subset sums are pairwise distinct, giving $M(14) \le 8$. □

9 Reproducibility and limitations

The repository separates fast checks from the decisive computation. The command `./test.sh` verifies every upper-bound certificate with two implementations, rejects broken controls, runs direct

small-instance comparisons, and exercises the Rust controls when the compiler is available. The scripts `enumerate/run_8x14_audit.sh` and `enumerate/run_8x14_rust_audit.sh` execute all ten partitions and write immutable-style bundles containing source, binary, environment, complete logs, structural checks, and SHA-256 manifests.

For the next unresolved case, $\beta(Q_{16})$, the artifact includes a portable nine-row extension of the C enumerator and a deterministic frontier-sampling driver, checked against all smaller controls and against the canonical form of the known 9×15 certificate. We do not report a runtime prediction, because the work is highly uneven across frontier branches. In a bounded pilot over the 66,547 canonical depth-six prefixes, the completed sample subtrees ranged from 2 to 30,964 nodes while others remained unfinished at the time limit; the driver therefore withholds a total-work estimate rather than extrapolating from the easier units. Establishing nonexistence at $n = 16$ would still require completing the entire frontier.

The public repository records the decisive node vectors, hashes, commands, and validation results. **[TODO — cut before final:]** Before journal submission, the complete C and Rust audit bundles should additionally be deposited in a durable public archive and assigned a DOI. This is an artifact-availability task, not a missing step in the enumeration itself, but it matters for long-term reproducibility.

The literature search located prior uses of the underlying hypercube symmetries, greedy and metaheuristic upper-bound searches, integer-programming checks, and the coin-weighing bridge [11, 12, 7, 9]. We found no prior report of the exact values at dimensions 14 or 15, nor of this exhaustive canonical-prefix computation. **[TODO — cut before final:]** Because absence claims are inherently fragile, the novelty statement should be checked once more by a domain expert and by the journal referee process.

10 Conclusion

Normalizing one row converts the hypercube metric-dimension question into a finite subset-sum enumeration with an explicit hypercube symmetry group. Incremental collision testing makes each extension local, and orderly generation under normalized row automorphisms removes noncanonical prefixes without sacrificing completeness. The resulting exhaustive computation establishes $\beta(Q_{14}) = \beta(Q_{15}) = 9$, while explicit certificates improve the upper bounds for Q_{24}, Q_{26}, and Q_{29} and give $M(14) = 8$. We also explored counterexample-guided abstraction refinement (CEGAR), but the present formulation did not improve upon the exhaustive method. Stronger abstractions, partition refinement (individualization-refinement), and full row-pivoting symmetry reductions remain productive directions for scaling. The natural next strengthening is an independently designed proof-producing enumeration; the natural next open parameter is $\beta(Q_{16})$.

Acknowledgments

The author thanks Victor S. Miller for helpful discussions and for his independent verification of the smaller terms of the sequence.

Disclosure of AI assistance

The exploratory search code and the orchestration of the computational campaign were developed with substantial assistance from large language models, principally Anthropic's Claude and OpenAI's Codex; the same assistance was used in drafting and revising this manuscript. The

mathematical results do not rest on that assistance. Every certificate and every mathematical nonexistence claim is established by the standalone verifiers and the exhaustive enumerator, each of which can be read, recompiled, and re-run independently of any AI system; the heuristics that discovered the upper-bound matrices are separated from the finite checks that prove them. The author has reviewed the definitions, proofs, and claims and takes full responsibility for their correctness.

A Detecting-matrix certificates

Each block below displays a sign matrix row by row, with `+` denoting $+1$ and `-` denoting -1. By Lemma 2, the absence of a nonzero ternary kernel vector certifies the stated metric-dimension upper bound. We include the 9×15 matrix used in Theorem 1(i) as well as the three new certificates from Theorem 1(ii).

A.1 A 9×15 certificate

```
----++++++-++++
-++-+-++++-----
+-+++-+-----+++
++----++--+-+-+
-+-+---++--++--
+--------+---+-
++++-+-+-+-++-+
+++++-+-++++++-
+-+-++--+--++--
```

A.2 A 13×24 certificate

```
-+--+-++-+-++-+-+-+--
+-+--++++---++++++--++-
--+-+---++-+-+-++-++-+-
+++--+-+++----++++++++++
-+-+-+--++-++-+--+-+-++-
+-++++-+++--+++++--+++-+-
--++-+++++-+-+-+-+--+++-
++++++++++----++--+--++-
-+--+-++-++--+-+-+-+-++-
+-+--++++-++--+---+++-+-
--+-+---+++-+-++-+--+++-
+++--+-++++++++--+--+++
+-++++-+++++--++++++--++-
```

A.3 A 14×26 certificate

```
++++++++-++++++++++++++++
--+--+-+-+-+-+-++-++-++-++
-+-+--+-++-+-++-+-+-++++++
++++---+++--+--++++++++++-
--+-+-+-++-+-+-+--+-++++
+--++--+++--++++--++++-+++
-+-+++-+++-+--+--+-+-+++++
++++++++++--+--+--+-++++++
--+--+-+-++-+-+--+--+-++++
+--+-++++-+++--+--++++-+++
-+-+--+-+++-++-+-+-+-+++++
++++---+++++++++--+-++++++
--+-+-+-+++-+-+-+-++-+++++
+--++--++++++--++++-++++++
```

A.4 A 15×29 certificate

```
+++++++++-++++++++++++++++++++
--+--+-+-+-+-+-+-+-++-+-+-++-
+--+--++++++--++++-+++--++++
-+-+-+--+-++-+--+-+++-++-++++
++++--+--+++--+--++++++++++++
--+-+-+-+-++-+-+-+-+-+-+-++
+--++++--+++--++++++--+++++-+
-+-++-++-+++-+--+-++-+--+-+++
++++++++++--+--+++--+--++++
--+--+-+-+-++-+-+-++-+-+-++
+--+--+++++-+++--+-+--++++++-+
-+-+-+--+-+++-++-+++-+--+-+++
++++--+--++++++++++--+--++++
--+-+-+-+-+++-+-+-+++-+-+-+++
+--+++++--++++++--++++++--++++
```

References

[1] Number of vectors so that no two subset sums are equal. MathOverflow question 157634, `https://mathoverflow.net/questions/157634/number-of-vectors-so-that-no-two-subset-sums-are-equal`. Accessed 11 August 2026.

[2] David G. Cantor and W. H. Mills. Determination of a subset from certain combinatorial properties. *Canadian Journal of Mathematics*, 18:42–48, 1966.

[3] Shmulik Cohen. Crack the test: Detecting matrices for hypercubes. GitHub repository, `https://github.com/shmulc8/crack-the-test`, 2026. Source and certificates; archival DOI pending.

[4] Paul Erdős and Alfréd Rényi. On two problems of information theory. *Magyar Tud. Akad. Mat. Kutató Int. Közl.*, 8:229–243, 1963.

[5] I. A. Faradžev. Generation of nonisomorphic graphs with a given distribution of the degrees of vertices. In *Algorithmic Studies in Combinatorics*, pages 11–19. Nauka, Moscow, 1978. In Russian.

[6] Frank Harary and Robert A. Melter. On the metric dimension of a graph. *Ars Combinatoria*, 2:191–195, 1976.

[7] Alain Hertz. An IP-based swapping algorithm for the metric dimension and minimal doubly resolving set problems in hypercubes. *Optimization Letters*, 14:355–367, 2020.

[8] Marijn J. H. Heule, Oliver Kullmann, and Victor W. Marek. Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer. In *Theory and Applications of Satisfiability Testing (SAT 2016)*, volume 9710 of *Lecture Notes in Computer Science*, pages 228–245, 2016.

[9] Changhong Lu and Qingjie Ye. A bridge between the minimal doubly resolving set problem in (folded) hypercubes and the coin weighing problem. *Discrete Applied Mathematics*, 309:147–159, 2022.

[10] Brendan D. McKay. Maximum dissociated sets of 0–1 vectors. Combinatorial data catalogue, `https://users.cecs.anu.edu.au/~bdm/data/dissociated.html`. Catalogue for dimension 7 contains 118,485 equivalence classes; accessed 11 August 2026.

[11] Nenad Mladenović, Jozef Kratica, Vera Kovačević-Vujčić, and Mirjana Čangalović. Variable neighborhood search for metric dimension and minimal doubly resolving set problems. *European Journal of Operational Research*, 220(2):328–337, 2012.

[12] Nebojša Nikolić, Mirjana Čangalović, and Igor Grujičić. Symmetry properties of resolving sets and metric bases in hypercubes. *Optimization Letters*, 11:1057–1067, 2017.

[13] OEIS Foundation Inc. The on-line encyclopedia of integer sequences, a303735. `https://oeis.org/A303735`. Accessed 11 August 2026.

[14] Ronald C. Read. Every one a winner or how to avoid isomorphism search when cataloguing combinatorial configurations. In *Annals of Discrete Mathematics*, volume 2, pages 107–120. North-Holland, 1978.

[15] András Sebő and Eric Tannier. On metric generators of graphs. *Mathematics of Operations Research*, 29(2):383–393, 2004.

[16] Peter J. Slater. Leaves of trees. In *Proceedings of the Sixth Southeastern Conference on Combinatorics, Graph Theory, and Computing*, volume 14 of *Congressus Numerantium*, pages 549–559. Utilitas Mathematica Publishing, 1975.

[17] Yong Kiam Tan, Marijn J. H. Heule, and Magnus O. Myreen. cake_lpr: Verified propagation redundancy checking in CakeML. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2021)*, volume 12652 of *Lecture Notes in Computer Science*, pages 223–241, 2021.